

PROYECTO: PRESUPUESTO MENSUAL PERSONAL

1. DESCRIPCIÓN GENERAL

1.1 Introducción

El presente proyecto consiste en el desarrollo de un sistema completo de gestión de presupuesto personal que permita a un usuario planificar, controlar y analizar sus finanzas personales de manera efectiva. El sistema debe gestionar ingresos, gastos, obligaciones mensuales y metas de ahorro a través de una aplicación web con reportería integrada.

1.2 Objetivo General

Aplicar los conocimientos adquiridos en la asignatura de Fundamentos de Sistemas de Bases de Datos mediante el diseño, implementación y despliegue de una solución completa de gestión financiera personal.

1.3 Alcance del Sistema

El sistema deberá permitir:

- Gestionar usuarios con información básica de perfil
- Definir presupuestos con vigencia temporal (de año/mes hasta año/mes)
- Clasificar ingresos, gastos y ahorros mediante categorías y subcategorías
- Registrar obligaciones fijas mensuales (servicios, deudas, seguros)
- Registrar transacciones individuales clasificadas por categoría
- Visualizar reportes analíticos y gráficos de gestión financiera

2. MODELO DE DATOS DETALLADO

2.1 Descripción de Entidades

USUARIO

Representa la persona que gestiona su presupuesto personal en el sistema.

Atributos:

- **Identificador único del usuario**
- **Nombre(s) del usuario**
- **Apellido(s) del usuario**
- **Correo electrónico (único)**
- **Fecha de registro en el sistema**
- **Salario mensual base (ingreso mensual regular)**
- **Estado del usuario (activo/inactivo)**

Descripción funcional: Almacena la información básica del usuario que utilizará el sistema. El salario base servirá como referencia para la planificación presupuestal inicial.

PRESUPUESTO

Define el plan financiero para un período específico con vigencia determinada.

Atributos:

- **Identificador único del presupuesto**
- **Usuario propietario del presupuesto**
- **Nombre descriptivo del presupuesto**
- **Año de inicio de vigencia**
- **Mes de inicio de vigencia (1-12)**
- **Año de fin de vigencia**
- **Mes de fin de vigencia (1-12)**
- **Total de ingresos planificados**
- **Total de gastos planificados**
- **Total de ahorro/provisión planificado**
- **Fecha y hora de creación**
- **Estado del presupuesto (activo/cerrado/borrador)**

Descripción funcional: Representa el marco temporal y los montos globales del presupuesto. La vigencia permite gestionar presupuestos de corto plazo (1 mes), mediano plazo (trimestral, semestral) o largo plazo (anual). Incluye los totales presupuestados para las tres categorías principales: ingresos, gastos y ahorro.

Reglas de negocio:

- El año de fin debe ser mayor o igual al año de inicio
 - Si el año de fin es igual al año de inicio, entonces el mes de fin debe ser mayor o igual al mes de inicio
 - Idealmente, el ingreso total presupuestado debe ser mayor o igual a la suma de gastos y ahorros presupuestados
 - Solo puede existir un presupuesto activo por usuario en un período dado
-

CATEGORIA

Clasificación principal de los conceptos presupuestales (SIEMPRE tiene subcategorías).

Atributos:

- **Identificador único** de la categoría
- **Nombre** de la categoría
- **Descripción detallada** de la categoría
- **Tipo de categoría** (ingreso/gasto/ahorro)
- **Nombre del icono** para UI (opcional)
- **Color en formato hexadecimal** para UI (opcional)
- **Orden de presentación** en la interfaz

Descripción funcional: Define las categorías principales del presupuesto. Cada categoría se clasifica en uno de tres tipos: ingreso (salario, bonos, freelance), gasto (alimentación, servicios, transporte, educación) o ahorro (fondo de emergencia, inversión, metas específicas).

IMPORTANTE: Toda categoría debe tener al menos una subcategoría. Cuando se crea una categoría, automáticamente se debe crear una subcategoría "General" o "Otros" mediante un trigger para garantizar esta regla.

Ejemplos de categorías:

- **Ingresos:** Salario Principal, Ingresos Adicionales, Bonificaciones, Freelance
- **Gastos:** Alimentación, Servicios Públicos, Transporte, Educación, Salud, Entretenimiento, Vivienda, Vestuario, Seguros
- **Ahorro:** Fondo de Emergencia, Inversiones, Metas Específicas, Provisiones

Reglas de negocio:

- **TODA categoría DEBE tener al menos una subcategoría (regla obligatoria)**

- Al crear una categoría, un trigger debe crear automáticamente una subcategoría por defecto llamada "General" o con el mismo nombre de la categoría
 - No se permite registrar transacciones ni presupuestos directamente a la categoría, sino a sus subcategorías
-

SUBCATEGORIA

Clasificación secundaria que permite granularidad en el control presupuestal (OBLIGATORIA para toda categoría).

Atributos:

- Identificador único de la subcategoría
- Categoría padre a la que pertenece
- Nombre de la subcategoría
- Descripción detallada
- Indicador si está activa
- Indicador si es la subcategoría por defecto (la creada automáticamente)

Descripción funcional: Permite descomponer las categorías principales en conceptos más específicos para un control detallado. Toda categoría debe tener al menos una subcategoría. Cuando se crea una categoría nueva, automáticamente se crea una subcategoría "General" o con el mismo nombre de la categoría mediante un trigger.

Ejemplos de subcategorías:

- **Alimentación:** Supermercado, Restaurantes, Comida Rápida, Cafeterías, General
- **Servicios Públicos:** Energía Eléctrica, Agua, Gas, Internet, Teléfono, General
- **Transporte:** Combustible, Transporte Público, Mantenimiento Vehículo, Estacionamiento, General
- **Educación:** Colegiatura, Libros, Material Escolar, Cursos Online, General
- **Entretenimiento:** Cine, Streaming, Deportes, Eventos, General
- **Salario Principal:** Salario Base (subcategoría por defecto)

Reglas de negocio:

- Una subcategoría solo puede pertenecer a una categoría
- Las subcategorías heredan el tipo de su categoría padre
- Toda categoría debe tener al menos una subcategoría activa

- Al crear una categoría, se debe crear automáticamente una subcategoría por defecto

PRESUPUESTO_DETALLE

Asignación de montos presupuestados mensuales a nivel de subcategoría para un presupuesto específico.

Atributos:

- Identificador único del detalle
- Presupuesto al que pertenece
- Subcategoría presupuestada
- Monto mensual asignado
- Observaciones o justificación del monto

Descripción funcional: Representa la distribución detallada del presupuesto a nivel de subcategoría con un monto mensual fijo. Cada registro define cuánto se presupuesta MENSUALMENTE para una subcategoría específica durante TODA la vigencia del presupuesto.

IMPORTANTE - Interpretación del Monto Mensual: Si un presupuesto tiene vigencia de Enero a Marzo 2025, y se crea un detalle con:

- Subcategoría: "Supermercado"
- Monto mensual asignado: L. 4,000.00

Esto significa que durante CADA MES (Enero, Febrero y Marzo) se presupuestan L. 4,000.00 para supermercado. NO se crean 3 registros separados. Es un solo registro que aplica mensualmente durante toda la vigencia.

Ejemplos:

Presupuesto con vigencia: Enero-Marzo 2025

Detalles del presupuesto (UN registro por subcategoría):

- Subcategoría: Supermercado, Monto mensual: L. 4,000.00
(Aplica para Enero, Febrero y Marzo - cada mes tiene L. 4,000 presupuestados)
- Subcategoría: Restaurantes, Monto mensual: L. 2,000.00
(Aplica para Enero, Febrero y Marzo - cada mes tiene L. 2,000 presupuestados)
- Subcategoría: Energía Eléctrica, Monto mensual: L. 800.00
(Aplica para Enero, Febrero y Marzo - cada mes tiene L. 800 presupuestados)

Cálculo del monto ejecutado: Para calcular el monto ejecutado de una subcategoría en un mes específico, se suman las transacciones de esa subcategoría filtradas por año y mes:

Cálculo del porcentaje de ejecución:

$$\text{Porcentaje} = (\text{Monto ejecutado del mes} / \text{Monto mensual presupuestado}) * 100$$

Ejemplo para Supermercado en Enero:

- Monto presupuestado: L. 4,000.00 (del detalle)
- Transacciones de Enero: L. 3,200.00 (suma de TRANSACCION)
- Porcentaje: $(3,200 / 4,000) * 100 = 80\%$

Cálculo del total por categoría: Para obtener el monto total presupuestado MENSUALMENTE de una categoría, se suma el monto mensual de todas sus subcategorías:

- Total mensual Alimentación = Supermercado (4,000) + Restaurantes (2,000) = L. 6,000.00 por mes

Uso durante la vigencia: Las transacciones de Enero a Marzo utilizarán este mismo presupuesto detalle para:

- Calcular el porcentaje de ejecución mensual
- Generar alertas cuando se supere el 80% o 100% en cualquier mes
- Comparar el desempeño mes a mes
- Generar reportes de cumplimiento presupuestal

Reglas de negocio:

- Todo detalle de presupuesto DEBE tener una subcategoría asociada (obligatorio)
 - No se permite NULL en el campo de subcategoría
 - El monto mensual asignado es el mismo para todos los meses dentro de la vigencia del presupuesto
 - Si se desea un presupuesto diferente por mes, se debe crear un nuevo presupuesto para ese período
 - Los montos ejecutados se calculan dinámicamente mes por mes sumando las transacciones correspondientes
 - El porcentaje ejecutado se calcula dinámicamente para cada mes: $(\text{suma transacciones del mes} / \text{monto mensual}) * 100$
 - Para obtener el total mensual de una categoría, se suman los montos mensuales de todas sus subcategorías
-

OBLIGACION_FIJA

Gastos recurrentes mensuales con monto y fecha de vencimiento predecibles.

Atributos:

- **Identificador único** de la obligación
- **Usuario propietario**
- **Subcategoría** asociada
- **Nombre** de la obligación
- **Descripción detallada**
- **Monto fijo mensual**
- **Día del mes** de vencimiento (1-31)
- **Indicador si está vigente**
- **Fecha de inicio** de la obligación
- **Fecha de finalización** (NULL si es indefinida)

Descripción funcional: Representa los compromisos financieros recurrentes que el usuario debe cumplir mensualmente. Cada obligación está asociada a una subcategoría específica, la cual determina la categoría padre.

Ejemplos:

- Alquiler de vivienda: L. 5,000.00 - subcategoría "Alquiler" de categoría "Vivienda" - vencimiento día 5
- Energía eléctrica: L. 800.00 - subcategoría "Energía Eléctrica" de categoría "Servicios Públicos" - vencimiento día 15
- Préstamo personal: L. 2,500.00 - subcategoría "Préstamo Bancario" de categoría "Deudas" - vencimiento día 10
- Seguro de vida: L. 600.00 - subcategoría "Seguros" de categoría "Protección" - vencimiento día 20

Reglas de negocio:

- Toda obligación DEBE tener una subcategoría asociada (obligatorio)
 - Si tiene fecha de finalización, debe ser mayor que la fecha de inicio
 - Las obligaciones activas generan alertas 3 días antes del vencimiento
 - El sistema puede generar automáticamente transacciones para obligaciones fijas
 - Una obligación fija solo puede estar asociada a subcategorías de categorías de tipo "gasto"
-

TRANSACCION

Registro de todos los movimientos financieros reales (ingresos, gastos, ahorros).

Atributos:

- **Identificador único** de la transacción
- **Usuario** que registra la transacción
- **Presupuesto** al que se imputa
- **Año** de la transacción (para efectos de presupuesto y reportes)
- **Mes** de la transacción (1-12, para efectos de presupuesto y reportes)
- **Subcategoría** de la transacción
- **Obligación** asociada (si aplica)
- **Tipo de transacción** (ingreso/gasto/ahorro)
- **Descripción** del movimiento
- **Monto** de la transacción
- **Fecha** en que ocurrió el movimiento
- **Método de pago** (efectivo/tarjeta_debito/tarjeta_credito/transferencia)
- **Número de factura o recibo** (opcional)
- **Observaciones adicionales** (opcional)
- **Fecha y hora de registro** en el sistema

Descripción funcional: Representa todos los movimientos financieros reales del usuario. Es la fuente de datos para calcular el presupuesto ejecutado y generar los reportes analíticos. Cada transacción debe estar asociada a una subcategoría, la cual determina su categoría padre.

IMPORTANTE - Campos Año y Mes (Modificables por el Usuario):

Los campos año y mes son INDEPENDIENTES del campo fecha y tienen las siguientes características:

Propósito:

1. **Imputación presupuestal:** Determinan a qué mes presupuestal se imputa la transacción
2. **Flexibilidad contable:** Permiten registrar una transacción en un mes diferente al de su fecha real
3. **Reportes mensuales:** Facilitan agrupar y filtrar transacciones por período presupuestal
4. **Rendimiento:** Mejoran el desempeño de consultas con índices en año/mes

Funcionamiento en la Interfaz:

- Al crear una transacción, la interfaz sugiere año/mes basándose en la fecha ingresada
- **El usuario PUEDE modificar** estos campos si desea imputar la transacción a un mes diferente
- Esto permite flexibilidad contable y de gestión presupuestal

Validaciones requeridas:

- El año/mes debe estar dentro del rango de vigencia del presupuesto asociado
- El sistema debe advertir (pero no impedir) si año/mes es muy diferente de la fecha real

Ejemplos:

- Compra en supermercado: fecha 2025-01-15, año 2025, mes 1, subcategoría "Supermercado", L. 1,200.00
- Recibo de luz pagado tarde: fecha 2025-02-05, año 2025, mes 1, subcategoría "Energía Eléctrica", L. 850.00 (usuario la asigna a enero)
- Salario mensual: fecha 2025-01-01, año 2025, mes 1, subcategoría "Salario Base", L. 15,000.00
- Pago adelantado: fecha 2025-01-28, año 2025, mes 2, subcategoría "Alquiler", L. 5,000.00 (alquiler de febrero pagado en enero)

Reglas de negocio:

- Toda transacción DEBE tener una subcategoría asociada (obligatorio)
- El tipo debe coincidir con el tipo de la categoría padre de la subcategoría
- Si la transacción está asociada a una obligación, se debe vincular mediante el identificador de la obligación
- Si está vinculada a una obligación, debe usar la misma subcategoría de la obligación
- La fecha de transacción debe estar dentro del rango de vigencia del presupuesto asociado
- Los campos año y mes determinan el mes presupuestal al que se imputa la transacción
- Los campos año y mes pueden ser diferentes del año/mes de la fecha real (son modificables por el usuario)
- Los campos año y mes deben estar dentro del período de vigencia del presupuesto asociado

3. PROCEDIMIENTOS ALMACENADOS Y FUNCIONES REQUERIDOS

3.1 Procedimientos CRUD Básicos

El alumno debe implementar procedimientos almacenados para operaciones CRUD (Create, Read, Update, Delete) en las siguientes tablas principales:

Tabla USUARIO

- `sp_insertar_usuario` : Crea un nuevo usuario con estado activo por defecto
- `sp_actualizar_usuario` : Actualiza datos básicos del usuario
- `sp_eliminar_usuario` : Desactiva un usuario (cambio de estado a inactivo)
- `sp_consultar_usuario` : Obtiene toda la información de un usuario específico
- `sp_listar_usuarios()` : Lista todos los usuarios del sistema con su estado

Tabla CATEGORIA

- `sp_insertar_categoria` : Crea una categoría (el trigger creará automáticamente la subcategoría por defecto)
- `sp_actualizar_categoria` : Actualiza nombre y descripción de una categoría
- `sp_eliminar_categoria` : Elimina una categoría si no tiene subcategorías adicionales activas
- `sp_consultar_categoria` : Obtiene información completa de una categoría
- `sp_listar_categorias` : Lista todas las categorías de un usuario filtradas por tipo (opcional)

Tabla SUBCATEGORIA

- `sp_insertar_subcategoria` : Crea una nueva subcategoría
- `sp_actualizar_subcategoria` : Actualiza información de subcategoría
- `sp_eliminar_subcategoria` : Elimina una subcategoría si no está en uso en presupuestos o transacciones
- `sp_consultar_subcategoria` : Obtiene información completa de una subcategoría incluyendo su categoría
- `sp_listar_subcategorias_por_categoria` : Lista todas las subcategorías de una categoría específica

Tabla PRESUPUESTO

- `sp_insertar_presupuesto` : Crea un nuevo presupuesto con estado activo
- `sp_actualizar_presupuesto` : Actualiza información del presupuesto
- `sp_eliminar_presupuesto` : Elimina un presupuesto si no tiene transacciones asociadas
- `sp_consultar_presupuesto` : Obtiene información completa de un presupuesto con sus fechas y estado

- `sp_listar_presupuestos_usuario` : Lista presupuestos de un usuario filtrados por estado (opcional)

Tabla PRESUPUESTO_DETALLE

- `sp_insertar_presupuesto_detalle` : Crea un detalle de presupuesto con monto mensual
- `sp_actualizar_presupuesto_detalle` : Actualiza el monto mensual presupuestado
- `sp_eliminar_presupuesto_detalle` : Elimina un detalle de presupuesto
- `sp_consultar_presupuesto_detalle` : Obtiene información completa de un detalle incluyendo subcategoría y categoría
- `sp_listar_detalle_presupuesto` : Lista todos los detalles de un presupuesto con información de subcategorías

Tabla OBLIGACION_FIJA

- `sp_insertar_obligacion` : Crea una obligación fija mensual
- `sp_actualizar_obligacion` : Actualiza datos de la obligación
- `sp_eliminar_obligacion` : Desactiva una obligación (cambio de estado)
- `sp_consultar_obligacion` : Obtiene información completa de una obligación con su subcategoría
- `sp_listar_obligaciones_usuario` : Lista obligaciones de un usuario filtradas por estado activo/inactivo

Tabla TRANSACCION

- `sp_insertar_transaccion` : Registra una transacción nueva
- `sp_actualizar_transaccion` : Actualiza datos de una transacción
- `sp_eliminar_transaccion` : Elimina una transacción (verificar impacto en metas de ahorro)
- `sp_consultar_transaccion` : Obtiene información completa de una transacción
- `sp_listar_transacciones_presupuesto` : Lista transacciones de un presupuesto con filtros opcionales

3.2 Procedimientos de Lógica de Negocio

Estos procedimientos encapsulan la lógica más compleja del sistema:

- `sp_crear_presupuesto_completo(p_id_usuario, p_nombre, p_descripcion, p_periodo_inicio, p_periodo_fin, p_lista_subcategorias_json, p_creado_por)` : Crea un presupuesto y sus detalles en una sola transacción. El parámetro JSON contiene array de {id_subcategoria, monto_mensual}

- `sp_registrar_transaccion_completa(p_id_usuario, p_id_presupuesto, p_anio, p_mes, p_id_subcategoria, p_tipo, p_descripcion, p_monto, p_fecha, p_metodo_pago, p_creado_por)` : Registra una transacción y valida que esté dentro del período del presupuesto y que el año/mes sean válidos
- `sp_procesar_obligaciones_mes(p_id_usuario, p_anio, p_mes, p_id_presupuesto)` : Revisa todas las obligaciones activas del usuario y genera alertas para las que vencen en el mes especificado
- `sp_calcular_balance_mensual(p_id_usuario, p_id_presupuesto, p_anio, p_mes, OUT p_total_ingresos, OUT p_total_gastos, OUT p_total_ahorros, OUT p_balance_final)` : Calcula el resumen financiero de un mes sumando todas las transacciones por tipo
- `sp_calcular_monto_ejecutado_mes(p_id_subcategoria, p_id_presupuesto, p_anio, p_mes, OUT p_monto_ejecutado)` : Suma todas las transacciones de una subcategoría para un mes específico
- `sp_calcular_porcentaje_ejecucion_mes(p_id_subcategoria, p_id_presupuesto, p_anio, p_mes, OUT p_porcentaje)` : Calcula el porcentaje ejecutado comparando el monto ejecutado vs el monto mensual presupuestado
- `sp_cerrar_presupuesto(p_id_presupuesto, p_modificado_por)` : Marca un presupuesto como cerrado, valida que la fecha de fin haya pasado y genera un resumen de ejecución
- `sp_obtener_resumen_categoria_mes(p_id_categoria, p_id_presupuesto, p_anio, p_mes, OUT p_monto_presupuestado, OUT p_monto_ejecutado, OUT p_porcentaje)` : Calcula el resumen de una categoría sumando todas sus subcategorías para un mes específico

3.3 Funciones Requeridas

El alumno debe implementar al menos las siguientes funciones:

- `fn_calcular_monto_ejecutado(id_subcategoria, anio, mes)` : Retorna el monto ejecutado de una subcategoría en un mes específico sumando transacciones
- `fn_calcular_porcentaje_ejecutado(id_subcategoria, id_presupuesto, anio, mes)` : Retorna el porcentaje ejecutado comparando transacciones vs presupuesto
- `fn_obtener_balance_subcategoria(id_presupuesto, id_subcategoria, anio, mes)` : Retorna el balance disponible (presupuestado - ejecutado) de una subcategoría en un mes
- `fn_obtener_total_categoria_mes(id_categoria, id_presupuesto, anio, mes)` : Retorna el total presupuestado de una categoría en un mes sumando todas sus subcategorías
- `fn_obtener_total_ejecutado_categoria_mes(id_categoria, anio, mes)` : Retorna el total ejecutado de una categoría en un mes sumando transacciones de todas sus subcategorías

- `fn_dias_hasta_vencimiento(id_obligacion)` : Calcula los días restantes hasta el vencimiento de una obligación
- `fn_validar_vigencia_presupuesto(fecha, id_presupuesto)` : Verifica si una fecha está dentro de la vigencia del presupuesto
- `fn_obtener_categoria_por_subcategoria(id_subcategoria)` : Retorna el identificador de la categoría padre de una subcategoría
- `fn_calcular_proyeccion_gasto_mensual(id_subcategoria, anio, mes)` : Proyecta el gasto final del mes basado en el comportamiento actual (días transcurridos vs días totales)
- `fn_obtener_promedio_gasto_subcategoria(id_usuario, id_subcategoria, cantidad_meses)` : Retorna el promedio de gasto en una subcategoría de los últimos N meses

3.4 Triggers (OBLIGATORIOS)

El alumno DEBE implementar triggers para automatizar procesos críticos del sistema. Los triggers son parte fundamental del proyecto y no son opcionales:

Triggers obligatorios a implementar:

1. **Trigger para crear subcategoría por defecto al insertar una categoría:**
 - Se activa cuando se inserta una nueva categoría
 - Crea automáticamente una subcategoría llamada "General" o con el mismo nombre de la categoría
 - Marca esta subcategoría como "por defecto"
 - Garantiza que toda categoría tenga al menos una subcategoría

Triggers opcionales (adicionales):

- Validar que año/mes de la transacción estén dentro de la vigencia del presupuesto asociado
- Sugerir año/mes basados en la fecha al insertar (pero permitir que el usuario los modifique)
- Validar reglas de negocio antes de inserciones/actualizaciones
- Registrar auditoría de cambios en tablas críticas
- Prevenir eliminación de registros con dependencias activas
- Validar que el tipo de transacción coincida con el tipo de la categoría padre de la subcategoría

Nota importante: Todas las bases de datos en la lista soportan triggers. El alumno debe implementar los 4 triggers obligatorios mencionados arriba.

4. MODELO RELACIONAL Y BASE DE DATOS

4.1 Campos de Auditoría Obligatorios

IMPORTANTE: Todas las tablas generadas en la base de datos (ya sean entidades fuertes o débiles) deben incluir los siguientes 4 campos de auditoría:

- **creado_por:** Identificador del usuario que creó el registro
- **modificado_por:** Identificador del usuario que modificó el registro por última vez
- **creado_en:** Fecha y hora de creación del registro (timestamp)
- **modificado_en:** Fecha y hora de la última modificación del registro (timestamp)

Notas:

- Estos campos Sí deben incluirse en el Modelo Relacional y en los scripts DDL
 - Los campos deben ser actualizados automáticamente mediante triggers o valores por defecto del sistema
 - El campo `creado_por` puede referenciar al usuario del sistema o almacenar el nombre/id del usuario de la aplicación
 - El campo `creado_en` debe tener un valor por defecto de la fecha/hora actual al momento de inserción
 - Los campos `modificado_por` y `modificado_en` deben actualizarse en cada UPDATE
-

5. DATOS DE PRUEBA

5.1 Requisitos de Generación de Datos

Cada alumno debe generar datos de prueba realistas para **DOS MESES COMPLETOS** con las siguientes características:

5.2 Características de Realismo

Los datos deben simular un comportamiento financiero realista:

1. **Distribución temporal:** Las transacciones deben estar distribuidas a lo largo del mes (no todas el mismo día)
2. **Montos coherentes:** Los montos deben ser proporcionales y realistas para cada categoría
3. **Obligaciones puntuales:** Las obligaciones fijas deben tener transacciones registradas cerca de su fecha de vencimiento

- 4. **Variabilidad:** Los gastos variables deben mostrar variación entre meses
- 5. **Balance realista:** El usuario no debe gastar exactamente lo presupuestado (debe haber variaciones)

5.3 Herramientas Sugeridas para Generación de Datos

Opción 1: Mockaroo (<https://www.mockaroo.com/>)

Opción 2: Faker (Python)

Opción 3: Generatedata (<https://generatedata.com/>)

Opción 4: Script SQL de Generación Propia: El alumno puede crear sus propios procedimientos almacenados que generen datos aleatorios directamente en la base de datos.

6. ARQUITECTURA DE LA APLICACIÓN

6.1 Arquitectura General

El sistema debe seguir una arquitectura de tres capas:



6.2 Capa de Presentación (Frontend):

Web, Desktop: se permite uso de IA en el Front-End unicamente si este modulo es Web y utiliza algun framework como ser: Radzen, Angular, Vue.js, React o algun otro.

6.3 Capa de Negocio (Backend/API):

Opcional: WebApi o Logica como parte del código o en su defecto el alumno podra crear la logica como parte del mismo proyecto, no se permite uso de IA

6.4 Capa de Datos:

La lógica de negocio debe residir en la base de datos, se debe invocar procedimientos almacenados, funciones, no es permitido ejecutar SQL de forma directa, no se permite uso de IA

7. MÓDULO DE REPORTERÍA

7.1 Reportes Requeridos

El alumno debe crear **6 REPORTES OBLIGATORIOS** y deben ser parte de la aplicación desarrollada y exportarse a PDF

Reporte 1: Resumen Mensual de Ingresos vs Gastos

Objetivo: Visualizar la comparativa mensual entre ingresos, gastos y ahorros para evaluar el balance financiero.

Tipo de visualización: Gráfico de barras agrupadas o apiladas

Datos a mostrar:

- Mes y Año
- Total de Ingresos del mes
- Total de Gastos del mes
- Balance final (Ingresos - Gastos)

Filtros:

- Rango de fechas (mes/año desde - hasta)
- Usuario (si aplica)

Valor agregado: Permite identificar tendencias de ahorro o déficit mensual

Reporte 2: Distribución de Gastos por Categoría

Objetivo: Mostrar qué porcentaje del presupuesto total se destina a cada categoría de gasto (sumando sus subcategorías).

Tipo de visualización: Gráfico circular (pie/dona chart)

Datos a mostrar:

- Nombre de categoría
- Monto total gastado en la categoría (suma de todas sus subcategorías)
- Porcentaje respecto al total de gastos
- Número de transacciones en la categoría (sumando todas sus subcategorías)

Cálculo requerido:

- El monto de cada categoría se obtiene sumando el monto ejecutado de todas sus subcategorías
- Ejemplo: Total Alimentación = Supermercado + Restaurantes + Comida Rápida + ...

Filtros:

- Mes/Año específico
- Usuario

Valor agregado: Identifica en qué categorías generales se está gastando más dinero

Reporte 3: Análisis de Cumplimiento de Presupuesto por Categoría y Subcategoría

Objetivo: Comparar el presupuesto asignado vs el monto realmente gastado a nivel de categoría y subcategoría, identificando desviaciones.

Tipo de visualización: Gráfico de barras comparativas con indicadores de color + Tabla detallada

Datos a mostrar:

- Categoría (agrupador principal)
- Subcategoría (detalle)
- Monto Presupuestado (a nivel de subcategoría)
- Monto Ejecutado/gastado (a nivel de subcategoría)
- Diferencia (presupuestado - ejecutado)
- Porcentaje de ejecución

- Total por Categoría (suma de subcategorías)
- Indicador visual:
 - **Verde:** < 80% ejecutado
 - **Amarillo:** 80-100% ejecutado
 - **Rojo:** > 100% ejecutado

Estructura del reporte:

```
Categoría: Alimentación (Total: L. 6,000 presupuestado / L. 5,200 ejecutado)
- Supermercado: L. 4,000 / L. 3,800 (95%) [Amarillo]
- Restaurantes: L. 2,000 / L. 1,400 (70%) [Verde]

Categoría: Servicios Públicos (Total: L. 2,000 / L. 2,150)
- Energía Eléctrica: L. 800 / L. 950 (119%) [Rojo]
- Agua: L. 400 / L. 350 (88%) [Amarillo]
- Internet: L. 800 / L. 850 (106%) [Rojo]
```

Filtros:

- Mes/Año
- Usuario
- Tipo de categoría (ingreso/gasto/ahorro)
- Mostrar solo subcategorías o incluir totales por categoría

Valor agregado: Identifica categorías y subcategorías específicas donde se está excediendo el presupuesto, permitiendo un control detallado

Reporte 4: Estado de Obligaciones Fijas y Cumplimiento de Pagos

Objetivo: Monitorear el cumplimiento de pago de las obligaciones fijas mensuales.

Tipo de visualización: Tabla con indicadores de estado + Resumen gráfico

Datos a mostrar (Tabla):

- Nombre de la obligación
- Categoría
- Monto mensual
- Fecha de vencimiento
- Estado de pago (Pagado/Pendiente/Vencido)

- Días hasta vencimiento / días de retraso
- Fecha del último pago registrado

Datos a mostrar (Gráfico):

- Resumen: % de obligaciones pagadas, pendientes y vencidas

Filtros:

- Mes/Año
- Estado (todos/pagadas/pendientes/vencidas)
- Usuario

Indicadores visuales:

- Verde: Pagada
- Amarillo: Pendiente (faltan 3+ días)
- Naranja: Por vencer (faltan <3 días)
- Rojo: Vencida sin pago

Valor agregado: Evita olvidos y cargos por mora

8. ENTREGABLES DEL PROYECTO

8.1 Repositorio Git (OBLIGATORIO)

Todos los alumnos deben crear un repositorio Git para el proyecto con las siguientes características:

Plataforma: GitHub, GitLab o Bitbucket

Estructura sugerida repositorio:

```
proyecto-presupuesto-personal/
├── README.md # Descripción del proyecto
├── docs/
│   ├── ModeloRelacional.dbml # Modelo Relacional documentado
│   └── ... (Otros archivos de Documentación)
├── database/
│   ├── DDL/
│   │   └── 01_crear_tablas.sql
│   ├── procedimientos/
│   │   ├── crud_usuario.sql
│   │   ├── crud_categoria.sql
│   │   └── ... (otros procedimientos)
│   ├── funciones/
│   │   └── funciones.sql
│   ├── triggers/
│   │   └── triggers.sql
│   └── datos_prueba/
│       └── insertar_datos.sql
├── backend/
│   ├── src/
│   ├── package.json (o equivalente)
│   └── README.md
└── frontend/
    ├── src/
    ├── assets/
    └── README.md
```

Prácticas requeridas:

- Commits frecuentes con mensajes descriptivos
- Uso de ramas (al menos: `main` , `develop`)
- Archivo `.gitignore` apropiado
- Documentación inline en el código

8.2 Avances

Se reservará un tiempo en cada semana para revisar avances del proyecto, cada avance puede requerir una demostración del avance del proyecto.

9. SISTEMA DE EVALUACIÓN

9.1 Distribución General de Puntos: 100 puntos totales

Componente	Porcentaje
Modelo de Datos y Diseño	30%
Implementación de Base de Datos	30%
Backend (API)	15%
Reportería	10%
Frontend	10%
Documentación y Presentación	5%

9.2 Sistema de Defensa Práctica (50% del Proyecto)

IMPORTANTE: El 50% de la nota del proyecto debe ser defendido mediante una prueba práctica individual basada en el código y SQL del proyecto.

Funcionamiento:

1. **Nota del Proyecto:** El proyecto se evalúa inicialmente sobre 100 puntos según la rúbrica detallada

2. **División de la Nota:**

- **50% de la nota:** Se mantiene automáticamente como nota base del proyecto
- **50% de la nota:** Debe ser defendido en una prueba práctica individual
- *En caso que el alumno use IA o su proyecto tenga nota de 0 no aplica a defensa.*

3. **Prueba o Exposición de Defensa:**

- Es individual y presencial
- Se basa en el código SQL, procedimientos, triggers y lógica del proyecto
- El docente hará preguntas técnicas y solicitará explicaciones del código
- Puede incluir solicitudes de modificación de código en vivo
- Duración aproximada: 20-30 minutos por alumno

4. **Cálculo de Nota Final:**

Si nota de defensa $\geq$ 80%:

- Se mantiene el 100% de la nota del proyecto
- Fórmula: $\text{Nota Final} = \text{Nota Proyecto}$

Si nota de defensa < 80%:

- El 50% defendible se penaliza proporcionalmente
- Fórmula: $\text{Nota Final} = (\text{Nota Proyecto} \times 50\%) + (\text{Nota Proyecto} \times 50\% \times \text{Nota Defensa} / 100\%)$

Ejemplos de Cálculo:

Ejemplo 1 - Defensa exitosa:

Nota del Proyecto: 90%
 Nota de Defensa: 85% ($\geq 80\%$)
 Nota Final: 90% (se mantiene completa)

Ejemplo 2 - Defensa con penalización:

Nota del Proyecto: 90%
 Nota de Defensa: 70% ($< 80\%$)

Cálculo:

- 50% base: $90\% \times 50\% = 45\%$
- 50% defendido: $90\% \times 50\% \times 70\% / 100\% = 31.5\%$
- Nota Final: $45\% + 31.5\% = 76.5\%$

Contenido de la Defensa:

- Explicación del modelo de datos y decisiones de diseño
- Explicación de procedimientos almacenados específicos
- Explicación de triggers y su lógica
- Explicación de consultas SQL de los reportes
- Resolución de problemas planteados por el docente
- Justificación de decisiones técnicas

Objetivo de la Defensa: Garantizar que el alumno comprende completamente el código que entrega y que es capaz de:

- Explicar la lógica implementada
- Modificar el código según nuevos requerimientos
- Resolver problemas similares de forma independiente

- Demostrar dominio de SQL y bases de datos
-

Conclusión: Este proyecto está diseñado para que el alumno aplique de manera integral los conocimientos de bases de datos, desarrollo web y análisis de datos, con énfasis en la implementación de lógica de negocio a nivel de base de datos mediante procedimientos almacenados y la generación de reportería analítica para toma de decisiones.
